

CPF

CPF Specification 기술 명세

CPF Public Contract 의/ identity, state, error, integration, runtime 의미를 정확히 확인합니다.

대상 독자 Framework 개발자 · 통합 개발자 · Architect · 검수자

문서 버전 Harness v1.1.3 / 문서 Rev. 1

기준 Source master 8670f6c9b675e3d210576c843d826898c781f9f0

기준일 2026-08-25

목차

1. Module · Base · Context 경계.....	4
1.1 Module/Public/SPI/Internal.....	4
1.2 Base Contract.....	4
1.3 Context Contract.....	4
2. 거래 Identity 계약.....	5
2.1 Canonical Header.....	5
2.2 Runtime Instance.....	5
2.3 Operation ID.....	5
3. Result · Error 계약.....	6
3.1 Result.....	6
3.2 Business/Technical/Validation Error.....	7
3.3 HTTP Mapping.....	7
4. Framework Common · Persistence 계약.....	8
4.1 Framework Common.....	8
4.2 Persistence.....	8
4.3 Data Quality.....	8
5. Transaction · Idempotency · Reconcile 계약.....	9
5.1 Transaction.....	9
5.2 Idempotency.....	9
5.3 Reconcile.....	9
5.4 Async/Workflow.....	9
6. Domain · External Integration 계약.....	10
6.1 Domain Invocation.....	10
6.2 External Integration.....	11
6.3 AI Integration.....	11
7. Messaging · Notification · Cache · File 계약.....	12
7.1 Messaging.....	12
7.2 Notification.....	12
7.3 Cache.....	12
7.4 File/Archive/Tabular.....	12
8. Security · Approval · Audit 계약.....	13
8.1 Security.....	13
8.2 Approval/Audit.....	13
9. Batch · Gateway 계약.....	14
9.1 Batch.....	14
9.2 Gateway.....	14
10. DB · Runtime · Platform State 계약.....	15
10.1 DB Lifecycle.....	15

10.2 Runtime/Topology.....	15
10.3 Feature Flag/Platform State.....	15
11. Operations · OpenAPI · Public API.....	16
11.1 Operations.....	16
11.2 OpenAPI.....	17
11.3 Public API Reference.....	17
12. State · Compatibility · Acceptance.....	18
12.1 State Transition.....	18
12.2 Compatibility.....	18
12.3 Contract Acceptance.....	18

1. Module·Base·Context 경계

Module/Public/SPI/Internal · Base Contract · Context Contract 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

1.1 Module/Public/SPI/Internal

Public API 는 고객 Application 이 직접 사용할 안정 계약이고 SPI 는 확장 구현 계약입니다. Internal type 은 Public BOM · Generated Domain compile surface 에 노출하지 않습니다.

- 확장점에는 lifecycle · 호환성 · 실패 의미를 함께 정의합니다.
- Native Spring/OSS API 가 더 적합한 경우 escape hatch 를 유지합니다.

1.2 Base Contract

Base Contract 는 CPF Public 계약이 공통으로 가져야 할 입력 · 결과 · 실패 · 호환성 원칙을 정의합니다. Consumer 는 구현 class 가 아니라 이 계약의 안정된 의미에 의존합니다.

- Base Contract 의 이름 · 필수 값 · 실패 의미는 Public API 와 Runtime 에서 동일하게 해석되어야 합니다.
- 정상/실패/UNKNOWN 과 호환성 위반을 Consumer 가 구분할 수 있어야 합니다.

1.3 Context Contract

Context Contract 는 transaction/system/operation identity 와 사용자 · 보안 context 가 Same JVM/Remote 경계에서 어떻게 유지 · 검증되는지 정의합니다.

- Context field 는 Controller 진입 전 검증되고 Same JVM/Remote 경계에서 동일 의미로 전달되어야 합니다.
- 필수 Context 누락 · 불일치는 업무 Controller 실행 전에 실패해야 합니다.

표 1-1. Module · Base · Context 경계 - Owner 와 Module 책임 경계

Module 별 Owner 책임과 직접 참조 금지 경계를 확인합니다.

영역/Module	책임	소유/경계	주요 Consumer
cpf-core	Kernel 의미	Runtime 구현 금지	Starter/Owner Runtime
cpf-common	고객 업무 공통	기술 Kernel 금지	Business Domain
cpf-admin	플랫폼 운영	업무 원장 금지	운영자/Frontend
cpf-batch	Batch Runtime	업무 master 금지	Scheduler/Worker
cpf-gateway	Edge/Routing	업무 로직 금지	외부 Client/Domain

표 1-2. Module·Base·Context 경계 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API	언제 사용	핵심 입력/옵션	실패/주의
Module/Public/SPI/Internal	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Base Contract	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Context Contract	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

Source 길찾기 `cpf-core/src/main/java/com/cpf/core/api/context/CpfContext.java`

2. 거래 Identity 계약

Canonical Header · Runtime Instance · Operation ID 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

2.1 Canonical Header

업무 Online Transaction 은 Controller 실행 전에 Canonical System6 를 갖습니다. Transaction/Original 은 유지하고 System/Caller/Target/Operation 은 Hop 기준으로 CPF 가 확정합니다.

- Browser 가 Protected Header 를 보내도 신뢰하지 않고 Trusted Entry 에서 재구성합니다.
- 수신 System/Target/Operation 이 실제 Runtime/Handler 와 다르면 업무 Controller 를 호출하지 않습니다.

2.2 Runtime Instance

instanceId 는 현재 Process 를 식별하는 Runtime 축이며 System6 Header 에는 포함하지 않습니다. 기동 시 설정→환경변수→Hostname 순으로 한 번 확정하고 Process 수명 동안 유지합니다.

- localhost/unknown/Domain 명 같은 fallback 으로 READY 를 허용하지 않습니다.
- 동일 Host 의 동일 System 다중 Process 는 explicit instanceId 가 필요합니다.

2.3 Operation ID

operationId 는 Annotation, OpenAPI, Domain Client, Registry, ADM, Log/Trace 가 공유하는 안정 실행 정의 ID 입니다. 개별 실행 건의 executionId 와 분리합니다.

- Source 는 operation metadata 를, ADM Policy 는 enabled/allowlist/override 를 소유합니다.

- Discovery 에서 일시 미발견된 Operation 을 자동 삭제하지 않습니다.

표 2-1. 거래 Identity 계약 - Canonical System6 설정·검증 규칙

각 Header 의 설정 주체·변경 규칙·수신 검증을 확인합니다.

Header	설정 주체	변경 규칙	검증/주의
X-Transaction-Id	CPF	최초 생성 후 불변	거래 전체 correlation
X-Original-System-Code	CPF Trusted Entry	최초 확정 후 불변	호출자가 임의 변경 금지
X-System-Code	CPF	Hop 마다 현재 처리 System	수신 Runtime 과 일치
X-Caller-System-Code	CPF	Hop 마다 직전 Caller	신뢰 System identity 사용
X-Target-System-Code	CPF	Hop 마다 Target	수신 Runtime 과 일치
X-Target-Operation-Id	CPF	Operation 마다 갱신	실제 Handler 와 일치

표 2-2. 거래 Identity 계약 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API	언제 사용	핵심 입력/옵션	실패/주의
Canonical Header	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Runtime Instance	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Operation ID	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

Source 길찾기 `cpf-starters/base/runtime/src/main/java/com/cpf/foundation/execution/api/CpfOperationCatalogRegistry.java`

3. Result·Error 계약

Result · Business/Technical/Validation Error · HTTP Mapping 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

3.1 Result

CPF 는 정상 결과와 Business/Technical/Validation 실패를 표준 Result/Error 의미로 구분합니다. 외부 Side Effect 결과가 확정되지 않으면 성공이나 실패로 추정하지 않고 UNKNOWN 으로 유지합니다.

- Business 실패는 정상 업무 판단과 구분해 코드/메시지를 전달합니다.
- Technical 실패는 retry 가능성·trace 식별자·운영 조치와 연결합니다.

3.2 Business/Technical/Validation Error

Business Error 는 업무 규칙상 거절, Validation Error 는 입력·계약 위반, Technical Error 는 Runtime/Provider 실패로 구분합니다. 각 분류의 retry·HTTP mapping·운영 조치가 달라야 합니다.

- Business 실패는 정상 업무 판단과 구분해 코드/메시지를 전달합니다.
- Technical 실패는 retry 가능성·trace 식별자·운영 조치와 연결합니다.

3.3 HTTP Mapping

External HTTP 호출은 timeout, deadline, credential, retry 조건을 호출 계약에 포함합니다. 응답을 받지 못한 상태에서는 원격 Side Effect 의 성공 여부를 추정하지 않습니다.

- retry 는 idempotent operation 또는 명시적 idempotency 계약에서만 허용합니다.
- credential·secret 을 URL/log/evidence 에 노출하지 않습니다.

표 3-1. Result·Error 계약 - 오류·복구 판단

오류 상태에서 Retry·복구·확인 순서를 판단합니다.

상태/오류	의미/원인	Retry·복구	확인/조치
Result	상태/원인 구분	조건부 Retry/Reconcile	trace·현재 상태 확인
Business/Technical/Validation Error	상태/원인 구분	조건부 Retry/Reconcile	trace·현재 상태 확인
HTTP Mapping	상태/원인 구분	조건부 Retry/Reconcile	trace·현재 상태 확인

표 3-2. Result·Error 계약 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API	언제 사용	핵심 입력/옵션	실패/주의
Result	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Business/Technical/Validation Error	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
HTTP Mapping	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

4. Framework Common·Persistence 계약

Framework Common · Persistence · Data Quality 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

4.1 Framework Common

Framework Common 계약은 Logging, Validation, Code/Parameter/Message 등 반복 기능의 Public Surface 와 Starter wiring 경계를 정의합니다.

- Common API 이름과 설정 key 는 Starter/Consumer 가 같은 계약을 사용하도록 유지합니다.
- 공통 기능 실패가 업무 오류로 잘못 변환되지 않도록 Error/Trace 의미를 유지합니다.

4.2 Persistence

Persistence 는 JDBC/MyBatis/JPA 중 업무와 팀 표준에 맞는 Provider 를 선택하면서 Repository·Paging·Bulk·Lock 의 공통 계약을 유지합니다.

- Domain 은 다른 Owner 의 Repository/DB 를 직접 조회하지 않습니다.
- Bulk/Paging 은 transaction 범위·lock·index·memory pressure 를 함께 검토합니다.

4.3 Data Quality

Data Quality 는 검증 Rule, 판정 Decision, 정정 흐름을 분리합니다. 단순 validation error 와 운영 정정이 필요한 품질 이슈를 같은 상태로 취급하지 않습니다.

- Rule 결과는 원 데이터와 추적 가능한 식별자를 남깁니다.
- 정정은 권한·이력·재검증 기준과 연결합니다.

표 4-1. Framework Common·Persistence 계약 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API	언제 사용	핵심 입력/옵션	실패/주의
Framework Common	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Persistence	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Data Quality	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

5. Transaction·Idempotency·Reconcile 계약

Transaction · Idempotency · Reconcile 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

5.1 Transaction

Transaction 계약은 local atomicity, propagation, idempotency 와 remote Side Effect 의 결과 확정/UNKNOWN 경계를 분리합니다.

- Transaction mode 와 propagation 의미는 annotation/config/runtime 에서 동일하게 해석되어야 합니다.
- remote 결과가 불명확하면 local rollback 여부와 별개로 UNKNOWN/Reconcile 경로를 사용합니다.

5.2 Idempotency

Idempotency 는 동일 의미의 요청이 재전송돼도 Side Effect 가 한 번만 확정되도록 durable 상태로 관리합니다. 메모리 flag 만으로 다중 인스턴스 중복을 막지 않습니다.

- key scope 와 결과 보존 기간을 업무 Operation 과 맞춥니다.
- 처리 중 Process Kill 후에도 재시도가 같은 결과를 재사용하거나 안전하게 복구해야 합니다.

5.3 Reconcile

UNKNOWN 은 결과를 확정할 Evidence 가 부족한 정상적인 운영 상태입니다. Reconcile 은 외부기관·DB·Broker 의 실제 결과를 조회해 상태를 확정하거나 manual review 로 넘깁니다.

- UNKNOWN 에서 무조건 재호출하면 중복 Side Effect 가 생길 수 있습니다.
- Reconcile 결과·actor·reason·시각을 추적 가능하게 남깁니다.

5.4 Async/Workflow

Async/Workflow 계약은 비동기 작업의 execution identity, 상태 전이, 재시도·중복 방지와 completion/reconcile 의미를 정의합니다. 요청 접수와 최종 완료를 같은 상태로 취급하지 않습니다.

- 변경 전 diff 와 ownership 을 확인합니다.
- 삭제가 필요한 경우 exact manifest/승인 절차를 거쳐 Generated/User-owned 경계를 보존합니다.

표 5-1. Transaction·Idempotency·Reconcile 계약 - 상태와 다음 처리

현재 상태에서 허용되는 다음 처리와 금지를 확인합니다.

상태	발생/조건	다음 처리	금지
Transaction	계약 조건	허용 transition	근거 없는 상태 덮어쓰기
Idempotency	계약 조건	허용 transition	근거 없는 상태 덮어쓰기
Reconcile	계약 조건	허용 transition	근거 없는 상태 덮어쓰기
Async/Workflow	계약 조건	허용 transition	근거 없는 상태 덮어쓰기

표 5-2. Transaction·Idempotency·Reconcile 계약 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API	언제 사용	핵심 입력/옵션	실패/주의
Transaction	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Idempotency	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Reconcile	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Async/Workflow	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

Source 길찾기 `cpf-starters/base/runtime/src/main/java/com/cpf/foundation/workflow/CpfWorkflow.java`

6. Domain·External Integration 계약

Domain Invocation · External Integration · AI Integration 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

6.1 Domain Invocation

업무 Source 는 배치 방식에 따라 local/remote 분기를 작성하지 않습니다. 공식 Domain Invocation 계층이 Same JVM 과 Remote route 를 선택하면서 동일한 operation·context·error 의미를 유지합니다.

- Same JVM 은 CpfContext 계열 논리 Context 를 사용하고 self-HTTP 를 하지 않습니다.
- Remote 는 System6, timeout/deadline, retry, trace 와 UNKNOWN/reconcile 계약을 경계에서 적용합니다.

6.2 External Integration

External Integration 계약은 target/operation, timeout/deadline, credential, retry 가능 조건과 원격 결과의 확정/UNKNOWN 의미를 정의합니다.

- Integration operation 과 target identifier 는 client, trace 와 Provider runtime 에서 동일하게 연결되어야 합니다.
- timeout/응답 유실은 원격 Side Effect 를 추정하지 않고 retry 가능 여부와 Reconcile 을 분리합니다.

6.3 AI Integration

AI 연계를 Provider 호출 자체보다 policy, risk, telemetry 와 결과 확정 의미를 함께 다룹니다. 외부 AI 응답 실패·timeout·UNKNOWN 을 일반 업무 성공으로 감추지 않습니다.

- 민감정보 전송·보존 정책을 Provider 경계에서 적용합니다.
- model/provider 변경은 compatibility 와 fallback 정책을 별도로 검증합니다.

표 6-1. Domain·External Integration 계약 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API		언제 사용	핵심 입력/옵션	실패/주의
Domain Invocation	해당 작업 수행		Public API/Config	실패·UNKNOWN 확인
External Integration	해당 작업 수행		Public API/Config	실패·UNKNOWN 확인
AI Integration	해당 작업 수행		Public API/Config	실패·UNKNOWN 확인

표 6-2. Domain·External Integration 계약 - Same JVM 과 Remote 계약 비교

배포 방식이 달라도 유지해야 할 호출 계약을 비교합니다.

관점		Same JVM	Remote	공통 계약
호출 경로	In-process route		Registry+transport	Domain Invocation
Context	CpfContext		System6 serialize	동일 논리 Context
실패	직접 예외/Result		timeout·network	표준 Error/UNKNOWN
관측	동일 trace		hop trace	transaction/operation 유지

Source 길찾기 `cpf-starters/integration/ai/src/main/java/com/cpf/integration/ai/CpfAiRouter.java`

7. Messaging·Notification·Cache·File 계약

Messaging · Notification · Cache 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

7.1 Messaging

Messaging 은 Broker 종류와 무관하게 schema, producer/consumer context, duplicate, DLQ 와 replay 를 함께 관리합니다. 전송 성공과 업무 처리 성공을 같은 상태로 보지 않습니다.

- Outbox/Inbox 를 사용할 때 DB commit 과 publish/consume 상태를 분리합니다.
- Replay 는 idempotency 와 ordering/partition 영향을 확인한 뒤 수행합니다.

7.2 Notification

Notification 은 발송 요청, Provider 결과, Receipt, Reconcile 상태를 분리합니다. 요청 수락만으로 최종 전달 성공을 확정하지 않습니다.

- Provider 별 message/receipt ID 를 내부 execution 과 연결합니다.
- UNKNOWN/지연 상태는 중복 발송 전에 조회·Reconcile 합니다.

7.3 Cache

Cache 는 get/put/evict 뿐 아니라 TTL, invalidation, multi-instance 전파와 stampede 를 함께 설계합니다. Provider 장애 시 원장 DB 와의 일관성 우선순위를 명확히 합니다.

- Cache 는 원장 Owner 를 대체하지 않습니다.
- serialization/provider conflict 와 stale data 를 실패 시나리오에 포함합니다.

7.4 File/Archive/Tabular

File 기능은 Attachment, Object Storage, Archive/Checksum/Tabular 처리를 선택형 Capability 로 제공합니다. 파일 원문과 metadata, scan/검증 상태를 분리합니다.

- checksum·content type·size·inspection 결과를 처리 전 검증합니다.
- Object Storage 장애/partial upload 는 reconcile/cleanup 경로를 가집니다.

표 7-1. Messaging·Notification·Cache·File 계약 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API	언제 사용	핵심 입력/옵션	실패/주의
Messaging	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

기능/API	언제 사용	핵심 입력/옵션	실패/주의
Notification	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Cache	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
File/Archive/Tabular	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

Source 길찾기 [cpf-starters/file](#)

8. Security·Approval·Audit 계약

Security · Approval/Audit 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

8.1 Security

Security 는 Authentication 이후 Authorization/Permission 을 분리해 평가하고 Session/Token lifecycle 을 Runtime 경계에 맞춥니다. 관리 기능과 업무 기능은 서로 다른 권한 Surface 를 가집니다.

- Protected Header 와 사용자 credential 을 동일 identity 로 취급하지 않습니다.
- 401/403 은 원인을 구분하고 실패도 audit/trace 와 연결합니다.

8.2 Approval/Audit

위험 운영 조치는 Permission 만으로 끝내지 않고 reason, 필요 시 approval, 실행 결과와 audit 를 연결합니다. Break-glass 는 일반 권한 우회가 아니라 별도 통제된 비상 절차입니다.

- 승인자와 실행자의 SoD 를 적용할 수 있어야 합니다.
- 실행 실패도 승인/요청/결과 trace 에서 누락하지 않습니다.

표 8-1. Security·Approval·Audit 계약 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API	언제 사용	핵심 입력/옵션	실패/주의
Security	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Approval/Audit	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

9. Batch·Gateway 계약

Batch · Gateway 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

9.1 Batch

Batch 계약은 Job/Execution/Step identity, 상태 전이, checkpoint, lease/fencing 과 Restart · Rerun · Reprocess · Reconcile 의미를 정의합니다.

- Job/Execution/Step identifier 는 Control Plane, Worker 와 운영 조회에서 같은 실행을 가리켜야 합니다.
- partial failure 와 UNKNOWN execution 은 성공 대상과 분리하고 허용 복구 동작을 명시합니다.

9.2 Gateway

Gateway 는 외부 진입 · trust normalization · routing · rate/resilience 를 담당하는 Optional Edge 입니다. 업무 로직과 Business DB 는 Gateway 에 두지 않습니다.

- Gateway 없이 배포하는 topology 도 공식 Domain 계약을 유지합니다.
- Route 변경은 health/drainng/canary/rollback 과 함께 운영합니다.

표 9-1. Batch · Gateway 계약 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API		언제 사용	핵심 입력/옵션	실패/주의
Batch	해당 작업 수행		Public API/Config	실패 · UNKNOWN 확인
Gateway	해당 작업 수행		Public API/Config	실패 · UNKNOWN 확인

표 9-2. Batch · Gateway 계약 - 역할과 장애 경계

각 역할의 책임과 장애 시 영향을 구분합니다.

역할	책임	소유/경계	장애 영향
Control Plane	정의 · 제어 · 상태	제어 Owner	명령/상태 불일치
Scheduler	Trigger · 할당	실행 계획	중복 scheduling
Worker	Job 실행	execution	Process kill

역할	책임	소유/경계	장애 영향
Center-Cut	대상 분할 · 조정	대량 실행	부분 실패
Agent	원격 실행 · heartbeat	instance/lease	stale agent

Source **길찾기** `cpf-gateway/src/main/java/com/cpf/gateway/api/CpfGatewayRoute.java`

10. DB·Runtime·Platform State 계약

DB Lifecycle · Runtime/Topology · Feature Flag/Platform State 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

10.1 DB Lifecycle

DB 변경은 Canonical Source→Fresh Init→Vendor Render→Migration→Seed→Upgrade→Rollback/Recovery→Runtime Query→Test 를 하나의 Lifecycle 로 닫습니다.

- DDL 한 파일 변경만으로 완료 처리하지 않습니다.
- Rollback 이 위험하거나 불가능하면 명시적 recovery 절차와 Evidence 를 남깁니다.

10.2 Runtime/Topology

Runtime/Topology 계약은 systemCode, instanceId, discovery/route 와 Same JVM · Remote · 다중 인스턴스에서 유지해야 할 identity/호출 의미를 정의합니다.

- systemCode · instanceId · target identity 는 discovery, health 와 trace 에서 같은 Runtime 을 가리켜야 합니다.
- instance/route 장애 시 다른 instance 로 전환해도 transaction/operation 의미와 실패 추적이 유지되어야 합니다.

10.3 Feature Flag/Platform State

Config 와 운영 State 는 Source 기본값, environment/profile, 정책/override 의 소유권과 우선순위를 분리합니다. Dynamic 변경은 적용 대상 · version · reason · rollback 가능성을 추적합니다.

- secret 값을 일반 property dump 나 ADM 화면에 노출하지 않습니다.
- 동적 변경은 다중 인스턴스 전파와 stale version 충돌을 검증합니다.

표 10-1. DB·Runtime·Platform State 계약 - DB Lifecycle 단계별 검증

DB 변경이 Fresh Init 부터 Runtime Query 까지 닫혔는지 확인합니다.

단계	Canonical	Vendor 영향	검증/복구
Fresh Init	Canonical schema	Vendor 별 DDL	깨끗한 초기화
Migration	변경 정의	문법/락 차이	upgrade 실행
Seed	공통 기준 데이터	DML 차이	재실행 안전성
Rollback/Recovery	역변경/복구	Vendor 별 절차	실패 복구 Evidence
Runtime Query	Repository 의미	SQL 차이	실거래 검증

표 10-2. DB·Runtime·Platform State 계약 - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API	언제 사용	핵심 입력/옵션	실패/주의
DB Lifecycle	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Runtime/Topology	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Feature Flag/Platform State	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

Source 길찾기 `cpf-starters/platform-operations/src/main/java/com/cpf/platform/operations/api/featureflag/CpfFeatureFlagOperations.java`

11. Operations·OpenAPI·Public API

Operations · OpenAPI · Public API Reference 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

11.1 Operations

Operations 계약은 transaction·operation·instance/Execution 식별자로 상태·Trace·위험 조치를 조회·제어하고 Permission·Reason·Approval·Audit 를 연결합니다.

- 운영 조회의 transactionId·operationId·instanceId 는 실제 Log/Trace/Runtime 상태와 일치해야 합니다.
- 위험 조치 실패와 부분 적용도 Audit/Trace 에서 최종 결과를 추적할 수 있어야 합니다.

11.2 OpenAPI

OpenAPI 는 Controller 계약의 operationId 와 오류/DTO 를 외부 Consumer Surface 로 고정합니다. Frontend 는 Generated Client 를 실제 화면 Consumer 에 연결하고 수기 중복 API wrapper 를 줄입니다.

- 401·403·404·409·429·500·503 상태를 화면/운영 흐름에서 구분합니다.
- Generated client 가 stale 하지 않도록 Source→OpenAPI→client→route coverage 를 검증합니다.

11.3 Public API Reference

Public API Reference 는 고객 Application 이 직접 사용할 타입과 operation 을 식별하고 실제 Starter/Consumer 경로를 연결합니다. Internal 구현 클래스는 Reference 대상에서 제외합니다.

- 확장점에는 lifecycle·호환성·실패 의미를 함께 정의합니다.
- Native Spring/OSS API 가 더 적합한 경우 escape hatch 를 유지합니다.

표 11-1. Operations·OpenAPI·Public API - 사용 기능/API 판단

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기능/API	언제 사용	핵심 입력/옵션	실패/주의
Operations	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
OpenAPI	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인
Public API Reference	해당 작업 수행	Public API/Config	실패·UNKNOWN 확인

표 11-2. Operations·OpenAPI·Public API - Source 길찾기

계약과 실제 Consumer 를 확인할 Source 위치를 찾습니다.

찾는 것	먼저 볼 위치	왜 보는가
계약/구현	cpf-starters/base/runtime/src/main/java/com/cpf/foundation/execution/api/CpfOperationCatalogRegistry.java	현재 Owner 와 Consumer 확인

Source 길찾기 cpf-core/src/main/java/com/cpf/core/api

12. State·Compatibility·Acceptance

State Transition · Compatibility · Contract Acceptance 을 중심으로 Consumer 가 따라야 할 계약과 상태 의미를 정리합니다.

12.1 State Transition

State Transition 계약은 허용 상태, 전이 조건, terminal/UNKNOWN 상태와 각 상태에서 허용되는 다음 행동을 정의합니다.

- State 이름과 transition 은 API 응답, 저장 상태와 운영 화면에서 동일하게 표현되어야 합니다.
- 허용되지 않은 transition 은 409 등 명시적 충돌로 거절하고 상태를 임의 덮어쓰지 않습니다.

12.2 Compatibility

Build/Compatibility 는 Java · Gradle · Spring baseline 과 Artifact Catalog 를 정본으로 맞춥니다. Publication/BOM/SBOM/verification 결과가 다른 Source snapshot 을 가리키지 않게 합니다.

- 현재 baseline 은 Java 25, Gradle 9.1.0, Spring Boot 4.1.0 입니다.
- 공개 Surface 변경은 consumer compile/runtime 과 generated output 까지 회귀 검증합니다.

12.3 Contract Acceptance

Contract Acceptance 는 문서/Interface 존재가 아니라 실제 Consumer→Runtime→Side Effect→Trace 가 계약대로 동작하고 실패 · 복구까지 일치하는지 판정합니다.

- Acceptance 는 Source 계약과 실제 Consumer/Runtime 결과가 같은 identifier 와 상태 의미를 갖는지 확인합니다.
- 실패 · 부분 실패 · UNKNOWN · 복구 시나리오가 실행되지 않았다면 Acceptance 완료로 판정하지 않습니다.

표 12-1. State · Compatibility · Acceptance - 상태와 다음 처리

현재 상태에서 허용되는 다음 처리와 금지를 확인합니다.

상태		발생/조건	다음 처리	금지
State Transition	계약 조건	허용 transition	근거 없는 상태 덮어쓰기	
Compatibility	계약 조건	허용 transition	근거 없는 상태 덮어쓰기	
Contract Acceptance	계약 조건	허용 transition	근거 없는 상태 덮어쓰기	

표 12-2. State·Compatibility·Acceptance - 지원·호환성 기준

현재 Source 의 지원 baseline 과 제약을 확인합니다.

항목		지원/기준	제약	확인
State Transition	현재 Source 지원	명시 설정/Provider	Source+Runtime 확인	
Compatibility	현재 Source 지원	명시 설정/Provider	Source+Runtime 확인	
Contract Acceptance	현재 Source 지원	명시 설정/Provider	Source+Runtime 확인	

Source 길찾기 gradle/cpf-stack.properties